

AGEX 개발 실행 인계문서

General-Purpose AI Operating System — Development Handoff & Execution Directive

1. 이 대화의 목적

이 대화는 **AGEX 실제 개발 전용 대화**다.

목적은 기존에 작성 완료된 AGEX 공식 문서와 Master Specification을 기준으로 실제 Repository를 구성하고, Canonical Schema를 작성하고, Runtime-Agent-Workflow-Model Gateway-IAM-Plugin-Knowledge-Memory-API-SDK-Console을 단계적으로 구현하는 것이다.

이 대화에서는 AGEX의 제품 철학을 다시 설계하지 않는다.

이미 확정된 문서를 **Single Source of Truth**로 사용하여 실제 코드를 구현한다.

2. AGEX 절대 정의

AGEX는 특정 고객, 특정 산업, 특정 업무를 위한 솔루션이 아니다.

AGEX는 다음을 핵심 Resource로 관리하는 **범용 AI Operating System**이다.

- Agent
- Workflow
- Knowledge
- Memory
- Plugin
- Model
- Runtime
- Marketplace

최종 목표:

AI가 생각하고, 기억하고, 판단하고, 협업하고, 실행할 수 있도록 하되 모든 행동을 Runtime-Security-Tenant-Permission-Policy-Governance-Cost-Observability 안에서 통제하는 범용 AI Operating System을 구현한다.

3. 다른 프로젝트와의 절대 분리

AGEX는 기존 다른 프로젝트와 완전히 분리한다.

다음 프로젝트의 기능, 용어, 데이터 모델, 화면, Workflow, 업무구조를 AGEX에 가져오지 않는다.

- 조원씨앤아이 여론조사 AI 플랫폼
- Pollinsight
- WBS Manager
- youthnet
- 정치 AI OS
- 특정 고객 프로젝트
- 특정 조사/설문/보고서 시스템

특정 산업 개념을 AGEX Core에 추가하지 않는다.

Application Layer에서만 특정 업무를 구현할 수 있다.

4. 개발 시작 전 필수 읽기 순서

Repository 작업을 시작하기 전에 반드시 다음 순서로 문서를 읽는다.

1. /AGENTS.md
2. /MASTER.md
3. /docs/INDEX.md
4. 현재 작업 관련 Domain Specification
5. /docs/GLOSSARY.md
6. /specs 관련 Canonical Contract

모든 문서를 매번 전부 읽을 필요는 없다.

docs/INDEX.md를 사용하여 현재 작업에 필요한 문서만 선택한다.

5. Source of Truth 우선순위

문서·Schema·코드가 충돌할 경우 다음 순서를 적용한다.

1. MASTER.md
2. Machine-readable Canonical Contracts (/specs)
3. Core Architecture / Runtime / Security / Multi-Tenant / Governance
4. Domain Specifications
5. API Specification
6. Development Roadmap / WBS / Development Plan
7. Guides
8. Existing Code

기존 코드가 명세와 다르다는 이유로 기존 코드를 기준으로 삼지 않는다.

명세와 구현이 충돌하면 먼저 충돌을 식별하고 공식 명세 기준으로 수정한다.

6. 공식 문서 세트

기존 AGEX 공식 문서는 다음 30종이다.

1. Product Vision
2. Product Philosophy
3. Product Architecture
4. Core Architecture
5. Runtime Architecture
6. Agent Framework
7. Workflow Engine
8. Knowledge Engine
9. Memory Engine

10. Plugin Framework
11. Marketplace
12. SDK
13. API Specification
14. Security Architecture
15. Multi-Tenant Architecture
16. Deployment Architecture
17. Governance
18. Product Roadmap
19. Development Roadmap
20. WBS
21. Development Plan
22. Resource Plan
23. Cost Estimation
24. Business Model
25. Technical White Paper
26. Product Documentation
27. Operations Guide
28. API Guide
29. Administrator Guide
30. Developer Guide

추가 공식 보완 명세:

- AGEX Master Specification
- Canonical Resource & Schema Specification
- Model Gateway / Model Router Architecture

- Identity & Access Management Specification
- Billing & Entitlement Architecture
- Development Harness
- Canonical Glossary

Agent Framework와 SDK는 **최신 재작성본**을 공식 기준으로 사용한다.

7. AGEX 절대 Architecture 원칙

다음 원칙은 구현 편의를 위해 변경하면 안 된다.

AI First

Agent Native

Workflow Native

Knowledge Native

Memory Native

Plugin Native

Runtime Centered

Model Agnostic

API First

Event Driven

Security by Design

Zero Trust

Multi-Tenant by Default

Observable by Default

Extensible by Default

Human Governed Autonomy

8. 반드시 구분해야 하는 개념

다음 개념을 절대 합치지 않는다.

Capability != Permission

Permission != Policy

Entitlement != Permission

Approval != Permission

Knowledge != Memory

Plugin != Tool

Plugin Definition != Plugin Installation

Installed != Enabled

Provider != Model

API Version != Resource Version

Version != Revision

Specification != Status

Resource Lifecycle != Execution Lifecycle

Execution != Operation

Queue != Execution Source of Truth

User != Tenant Membership

User Identity != Agent Identity != Execution Identity

9. Resource 기본 구조

AGEX의 주요 Resource는 기본적으로 다음 구조를 따른다.

Resource

├─ metadata

├─ specification

└─ status

공통 핵심:

id

kind

scope_type

tenant_id

version

revision

lifecycle_state

owner

labels

created_at

updated_at

10. Version과 Revision

공식 정의:

Revision

Draft 내부 변경 번호.

Version

Publish 시 생성되는 Immutable Specification.

따라서:

Published Version은 직접 수정하지 않는다.

수정이 필요하면 새로운 Version을 생성한다.

11. Platform Scope / Tenant Scope

Resource는 다음 Scope를 가진다.

PLATFORM

TENANT

TENANT Scope Resource는 반드시 Tenant Context를 가진다.

Tenant Context를 안전하게 결정할 수 없으면:

FAIL CLOSED

12. Tenant Context 전파

Tenant Context는 다음 전체 경로에서 유지되어야 한다.

API

→ Domain

→ Runtime

→ Queue

→ Worker

→ Database

→ Search

→ Vector

→ Cache

→ Event

→ Audit

Cross-Tenant Access는 기본 DENY다.

13. 검색 보안

다음은 금지한다.

Global Search

→ Top K

→ Tenant Filter

반드시:

Tenant / ACL Filter

→ Candidate Generation

→ Ranking

순으로 처리한다.

14. Runtime 절대 원칙

모든 Durable AI Execution은 AGEX Runtime을 통과한다.

Runtime 책임:

- Execution State
- Task
- Queue Dispatch

- Worker Lease
- Retry
- Timeout
- Cancellation
- Wait
- Checkpoint
- Recovery
- Reconciliation

Queue는 Source of Truth가 아니다.

Source of Truth는 Durable Execution Store다.

15. Runtime Delivery Semantics

Task Dispatch는 기본적으로:

AT-LEAST-ONCE

를 전제로 한다.

따라서 Side Effect Action은 반드시 다음 중 하나 이상을 지원해야 한다.

- Idempotency Key
- Deduplication
- Side-Effect Ledger
- External Idempotency

분산 시스템 전체를 "Exactly Once"라고 표현하지 않는다.

16. Runtime 공식 Execution State

CREATED

VALIDATING

QUEUED

RUNNING

WAITING

RETRYING

COMPLETED

FAILED

CANCELLED

TIMED_OUT

TERMINATED

Approval, User Input, Event Wait 등은 별도 waiting_reason으로 표현한다.

예:

state = WAITING

waiting_reason = APPROVAL

17. Agent 공식 정의

Agent는 Prompt Template이 아니다.

Agent는 다음을 포함하는 Versioned Resource다.

Goal

Instruction

Capability

Permission

Autonomy

Model Policy

Context Policy

Knowledge

Memory

Tools

Planning

Collaboration

Execution Policy

Evaluation

Model은 Action을 제안할 수 있지만 실제 실행 권한은 없다.

Model proposes

→ Runtime validates

→ Runtime executes

18. Agent Autonomy 공식 단계

L0 Response Only

L1 Plan Proposal

L2 Execute After Approval

L3 Restricted Autonomous Execution

L4 Continuous Policy-Based Execution

L5 Multi-Agent Autonomous Collaboration

Core MVP는 기본적으로 L0~L2까지 구현한다.

L3는 제한 Preview 가능.

L4/L5는 초기 Production MVP 범위가 아니다.

19. Agent 권한

Agent는 User Permission 전체를 자동 상속하지 않는다.

실제 Agent Permission:

Agent Permission

∩

Delegated Scope

∩

Tenant Policy

∩

Runtime Policy

Agent가 자신의 Permission, Tenant, Autonomy를 변경할 수 없다.

20. Multi-Agent

Agent-to-Agent 호출은 함수 호출이 아니다.

반드시:

Parent Execution

→ Child Execution

→ Agent B

형태로 관리한다.

Parent Permission 전체를 Child Agent에게 상속하지 않는다.

Delegation Depth와 Child Count는 제한한다.

21. Workflow 공식 정의

Workflow는 Durable Execution Graph다.

지원 핵심 Step:

AGENT

MODEL

PLUGIN

FUNCTION

CONDITION

SWITCH

PARALLEL

JOIN

LOOP

WAIT

EVENT_WAIT

APPROVAL

SUB_WORKFLOW

EMIT_EVENT

END

Workflow 상태를 Worker Process Memory에만 저장하지 않는다.

Loop/Retry는 반드시 Bounded해야 한다.

22. Knowledge와 Memory

Knowledge

Source-backed official information.

Memory

선택적으로 보존된 interaction/execution context.

Memory를 Knowledge로 자동 승격하지 않는다.

Memory

→ Candidate

→ Validation

→ Review

→ Approval

→ Knowledge

23. Plugin 공식 구조

Plugin Package

→ Capability

→ Action

→ Agent Tool Projection

Plugin은 Tool이 아니다.

Plugin Action은 최소 다음을 정의한다.

input_schema

output_schema

required_permissions

side_effect

timeout

idempotency

공식 Side Effect:

READ_ONLY

REVERSIBLE_WRITE

IRREVERSIBLE_WRITE

PRIVILEGED_ACTION

24. Plugin Credential

Credential은 Plugin Definition에 저장하지 않는다.

구조:

Tenant Plugin Installation

→ Credential Binding

→ Secret Reference

→ Secret Store

Agent Context에는 Secret을 넣지 않는다.

25. Model Gateway

모든 Model 호출은 Model Gateway를 통과한다.

금지:

Agent → Provider SDK

Workflow → Provider API

Application → Provider directly

허용:

Provider Adapter → Provider SDK

26. Model Routing 공식 우선순위

1. Required Capability
2. Data Classification
3. Platform Security Policy
4. Tenant Model Policy
5. Region / Residency
6. Provider Trust
7. Model State
8. Quality
9. Health / Capacity

10. Latency

11. Cost

Cost가 Security보다 앞설 수 없다.

Fallback에서도 전체 Security/Residency 검증을 다시 수행한다.

27. Provider Trust Class

공식:

PRIVATE

DIRECT_APPROVED

ENTERPRISE_APPROVED

AGGREGATOR

RESTRICTED

BLOCKED

BLOCKED Provider는 어떤 Fallback에서도 사용할 수 없다.

28. Data Classification

공식:

PUBLIC

INTERNAL

CONFIDENTIAL

RESTRICTED

여러 데이터가 섞이면 가장 높은 Classification을 기본 적용한다.

Agent가 Data Classification을 낮출 수 없다.

29. Governance Risk

공식:

LOW

MODERATE

HIGH

CRITICAL

Data Classification과 Risk를 혼용하지 않는다.

30. IAM

공식 Principal:

USER

SERVICE

WORKLOAD

AGENT

SYSTEM

User와 Membership은 분리한다.

Service Automation에는 사람 계정을 사용하지 않는다.

Authorization은 서버에서 수행한다.

Frontend 메뉴 Visibility는 Security Boundary가 아니다.

31. Security 우선순위

Platform Security Baseline

>

Environment Policy

>

Tenant Policy

>

Resource Policy

>

Agent / Workflow Instruction

>

User Instruction

Security DENY는 다음으로 Override할 수 없다.

- Approval
- Entitlement
- Agent reasoning
- Tenant preference
- Cost optimization

32. Secret 정책

다음에 Secret을 저장하지 않는다.

- Prompt
- Agent Instruction
- Memory
- Workflow Variable
- Plugin Definition
- General Log
- Audit

사용:

Secret Reference

→ Secret Store

→ Credential Broker

33. Raw Chain-of-Thought

Raw CoT는:

- 저장하지 않는다.
- API로 노출하지 않는다.
- Audit에 사용하지 않는다.
- Debugger 필수 데이터로 사용하지 않는다.

대신 구조화된 정보를 저장한다.

Plan Step

Selected Action

Tool Call

Knowledge Reference

Policy Decision

Evaluation

Failure Reason

Model Metadata

34. Marketplace

Marketplace는 실행하지 않는다.

공식 Commercial Flow:

Package

→ Listing

→ Offer

→ Order

→ Entitlement

→ Installation

Entitlement와 Permission은 다르다.

35. Billing

Commercial State와 Security State를 분리한다.

Billing Restricted != Security Suspended

Plan Upgrade가 IAM Permission을 자동 확대하지 않는다.

Downgrade 시 초과 Resource를 즉시 삭제하지 않는다.

36. API

공식 원칙:

Canonical Schema

→ API

→ SDK

→ CLI / Console

DB가 API Contract를 결정하지 않는다.

Console이 DB를 직접 조작하지 않는다.

37. SDK

공식 구조:

Canonical Schema

→ Generated Low-Level Client

→ High-Level SDK

초기 Tier 1:

- TypeScript
- Python

분리 SDK:

- Client SDK
- Plugin SDK
- Model Adapter SDK
- Testing SDK

38. Event

Event는 이미 발생한 사실이다.

ExecutionCompleted

AgentPublished

Command와 Event를 혼용하지 않는다.

공통 Envelope:

event_id

event_type

event_version

occurred_at

scope_type

tenant_id

actor

resource

correlation_id

data

39. Error Contract

공식 Error Category:

AUTHENTICATION

AUTHORIZATION

POLICY

VALIDATION

NOT_FOUND

CONFLICT

QUOTA

RATE_LIMIT

RUNTIME

PROVIDER

TIMEOUT

INTERNAL

Client가 Message String을 Parsing하게 하지 않는다.

Stable Error Code를 사용한다.

40. DB 원칙

초기 구현은 Modular Monolith를 허용한다.

하나의 RDBMS를 사용할 수 있다.

그러나 Domain Boundary는 유지한다.

금지:

Agent Module

→ Memory Table 직접 조회

허용:

Agent Service

→ Memory Contract

41. 초기 물리 Architecture

기본 권장:

Modular Monolith

+

Distributed Runtime Workers

처음부터 과도한 Microservices 분리를 하지 않는다.

Service 분리는 다음 조건에서만 검토한다.

- 독립 Scale
 - Security Boundary
 - Failure Isolation
 - Separate Deployment Lifecycle
 - Team Ownership
 - Specialized Infrastructure
-

42. 개발 순서

공식 순서:

Requirement

→ Specification

→ Canonical Schema

→ API/Event Contract

→ Implementation

→ Test

→ Documentation

→ Acceptance

코드부터 만들고 나중에 문서를 맞추지 않는다.

43. Public Contract 변경 절차

Resource/API/Event/Permission 변경 시:

1. 해당 Specification 확인
2. Canonical Schema 수정
3. Backward Compatibility 판단
4. API/Event Contract 수정
5. SDK Type 재생성
6. 코드 구현
7. Contract Test
8. 문서 수정

44. 새 개념 생성 규칙

다음 항목을 새로 만들기 전에 반드시 Canonical Specification을 검색한다.

- Resource
- Lifecycle State
- Permission
- Event
- Error Category
- Workflow Step
- Model Capability
- Plugin Side Effect

없다면 임의 구현하지 않는다.

명세 Gap을 먼저 기록한다.

45. Definition of Done

다음이 모두 완료되어야 한다.

Specification compliance

Implementation

Unit Tests

Contract Tests

Integration Tests

Tenant Isolation Tests

Security Tests

Observability

Documentation

Acceptance Evidence

AI 기능:

+ Evaluation

화면에서 동작하는 것만으로 완료가 아니다.

46. MVP 공식 범위

Core MVP:

- Tenant
- IAM
- Core Resource
- Runtime

- Agent L0-L2
- Workflow
- Model Gateway
- Basic Knowledge
- Basic Memory
- Plugin Core
- Security
- Audit
- Usage
- Observability
- API
- TypeScript/Python SDK
- Minimal Developer Console

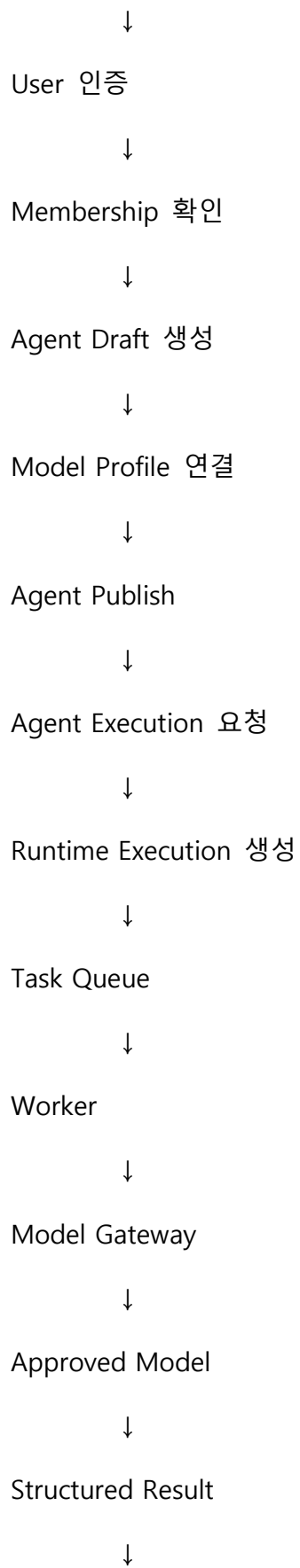
후순위:

- Public Marketplace
- Full Billing
- L4/L5
- Complex Multi-Agent
- Federation
- Self-Improvement
- Advanced Dynamic Workflow Generation

47. 첫 개발 Vertical Slice

첫 번째 구현 목표는 반드시 다음이다.

Tenant 생성



Execution Complete



Audit



Usage



Trace

이 Vertical Slice를 먼저 완성한다.

48. 두 번째 Vertical Slice

Agent

→ Knowledge Retrieval

→ Model

→ Read-Only Tool

→ Runtime Validation

→ Plugin

→ Result

→ Complete

49. 세 번째 Vertical Slice

Agent

→ Irreversible Action Candidate

→ Permission

→ Risk

→ Approval

- WAITING
 - Human Approves
 - Runtime Revalidation
 - Plugin Action
 - Complete
-

50. 네 번째 Vertical Slice

Workflow

- Agent
 - Condition
 - Approval
 - Plugin
 - Complete
-

51. 다섯 번째 Vertical Slice

Execution Running

- Worker Crash
- Lease Expire
- Runtime Reconciliation
- Redispatch
- Checkpoint Restore
- Complete

이것이 성공해야 Durable Runtime 구현이 완료된 것으로 인정한다.

52. Repository 초기 구성

가장 먼저 다음 구조를 만든다.

agex/

└─ AGENTS.md

└─ CLAUDE.md

└─ MASTER.md

|

└─ docs/

| └─ INDEX.md

| └─ GLOSSARY.md

| └─ constitution/

| └─ architecture/

| └─ domains/

| └─ supplemental/

| └─ planning/

| └─ business/

| └─ guides/

|

└─ specs/

| └─ schemas/

| └─ permissions/

| └─ events/

| └─ fixtures/

|

└─ adr/

└─ src/

└─ tests/

└─ tools/

53. 최초 Machine-Readable Schema 작성 순서

반드시 다음 순서로 진행한다.

1. ResourceMetadata
2. ResourceReference
3. PrincipalReference
4. TenantContext
5. ErrorEnvelope
6. EventEnvelope
7. RetryPolicy
8. TimeoutPolicy
9. Tenant
10. User
11. Membership
12. Role
13. ServiceIdentity
14. Execution
15. Task
16. Provider
17. Model
18. ModelProfile
19. Agent
20. Workflow

21. Plugin

22. Knowledge

23. Memory

24. Policy

25. Approval

54. 초기 개발 단계

Phase 0 — Foundation

구현:

- Repository
- Harness
- Canonical Schema
- Common Types
- Error
- Event
- Tenant Context
- Basic Auth

완료 조건:

- Schema Validation
- Type Generation
- Contract Test

55. Phase 1 — Tenant + IAM

구현:

- Tenant

- User
- Membership
- Role
- Permission
- Service Identity
- Session

완료 조건:

Tenant A Identity

→ Tenant B Resource

→ DENY

자동 테스트 통과.

56. Phase 2 — Runtime

구현:

- Execution
- Task
- Queue
- Worker
- Lease
- Retry
- Timeout
- Cancel
- Checkpoint
- Recovery

완료 조건:

Worker Crash Recovery.

57. Phase 3 — Model Gateway

구현:

- Provider Registry
- Model Registry
- Model Profile
- Provider Adapter
- Router
- Structured Output
- Usage
- Fallback

완료 조건:

Agent/Runtime이 Provider를 직접 알지 않고 Model 실행 성공.

58. Phase 4 — Agent

구현:

- Agent Resource
- Draft
- Publish
- Activate
- L0-L2
- Context Builder
- Action Candidate
- Model Integration

완료 조건:

첫 Vertical Slice.

59. Phase 5 — Workflow

구현:

- Graph
 - Step
 - Branch
 - Wait
 - Approval
 - Retry
 - Subworkflow
-

60. Phase 6 — Knowledge / Memory

구현:

- Knowledge Collection
 - Source
 - Retrieval
 - Provenance
 - Memory Proposal
 - Recall
 - Forget/Delete
-

61. Phase 7 — Plugin

구현:

- Plugin Definition
- Action
- Installation
- Credential Binding
- Permission
- Side Effect
- Runtime

62. Phase 8 — Developer Platform

구현:

- API
- SDK
- CLI
- Developer Console

63. 코딩 에이전트 작업 보고 형식

Codex/Claude는 중요한 작업 전 다음을 먼저 요약한다.

Governing Specification:

Affected Domain:

Affected Resource:

Schema Impact:

API/Event Impact:

Tenant/Security Impact:

Required Tests:

그 후 실제 구현한다.

64. 구현 중 명세 부족 발견 시

임의로 구현하지 않는다.

다음 형식으로 표시한다.

SPECIFICATION GAP

Domain:

Missing Contract:

Why Required:

Proposed Minimal Resolution:

Affected Files:

Architecture를 크게 바꾸지 않는 최소안을 제안한다.

65. 기존 코드와 명세 충돌 시

다음 순서를 따른다.

1. 충돌 발견
2. Governing Specification 확인
3. Existing Code 영향 분석
4. 명세가 명확하면 코드 수정
5. Migration 필요 여부 확인
6. Regression Test 추가

기존 코드가 이미 많다는 이유로 잘못된 Architecture를 유지하지 않는다.

66. 리팩터링 원칙

현재 Task와 관계없는 대규모 리팩터링을 하지 않는다.

단, 명세 준수를 위해 Domain Boundary 수정이 반드시 필요한 경우에는 수행할 수 있다.

변경 이유를 명확히 기록한다.

67. 테스트 우선순위

모든 중요 기능에 다음을 고려한다.

Unit

Contract

Integration

Tenant Isolation

Security

Failure Recovery

AI Evaluation

68. Security Critical Code

다음 코드는 반드시 강한 Review 대상이다.

- Authentication
- Authorization
- Tenant Isolation
- Runtime State Machine
- Task Lease
- Secret
- Model Routing
- Plugin Sandbox
- Database Migration

AI Coding Agent가 작성해도 Human Review를 전제로 한다.

69. Commit / PR 연계

가능하면 WBS/Requirement와 Commit/PR을 연결한다.

예:

AGEX-RT-REQ-014

Commit/PR Description:

Implements: AGEX-RT-REQ-014

Specification: Runtime Architecture §...

Schema: execution.schema.json

Tests: runtime recovery integration test

70. 최종 개발 목표

AGEX 구현은 기능 개수를 늘리는 것이 목적이 아니다.

최우선 목표:

Correct Contract

+

Durable Runtime

+

Strong Tenant Isolation

+

Explicit Security

+

Model Neutrality

+

Extensible Resource Architecture

이 기반이 완성된 후 Marketplace와 고도 자율 기능을 확장한다.

71. 이 대화에서의 첫 작업

이 문서를 읽은 후 가장 먼저 해야 할 작업은:

현재 AGEX Repository 상태를 확인하고, Repository가 없다면 공식 AGEX Repository 기본 구조와 Harness를 생성한 뒤 Phase 0 Canonical Schema Foundation을 구축하는 것.

구체적으로:

AGENTS.md

CLAUDE.md

MASTER.md

docs/INDEX.md

docs/GLOSSARY.md

specs/schemas/common/

specs/events/

specs/permissions/

tests/contracts/

부터 생성한다.

그다음 Common Schema:

ResourceMetadata

ResourceReference

PrincipalReference

TenantContext

ErrorEnvelope

EventEnvelope

RetryPolicy

TimeoutPolicy

를 작성하고 Validation Test를 만든다.

72. 첫 응답에 요구되는 행동

이 인계문서를 받은 새 개발 대화의 AI는 바로 코드를 추측해서 만들지 말고 먼저:

1. AGEX Repository 존재 여부 확인
2. 현재 폴더/파일 구조 확인
3. Git 상태 확인
4. 기존 AGENTS.md, CLAUDE.md, MASTER.md 존재 여부 확인
5. 기존 Source Stack 확인
6. 기존 구현과 AGEX 명세의 충돌 가능성 확인

후 현재 Repository 상태를 기준으로 Phase 0 실행계획을 제시하고 바로 구현을 시작한다.

불필요한 재기획이나 AGEX 개념 재설명은 하지 않는다.

73. 최종 지시

이제부터 AGEX 개발은 문서 중심 설계 단계가 아니라 Specification-Driven Implementation 단계다. 모든 코드는 AGEX Master Specification과 Canonical Contract를 구현하는 수단이며, 기존 구현이나 편의적 선택이 공식 Architecture를 변경해서는 안 된다.

항상 AGENTS.md → MASTER.md → INDEX.md → 관련 Domain Specification → Canonical Schema → Code → Test 순서로 작업한다.

AGEX는 범용 AI Operating System이며 특정 고객·산업·업무 프로젝트의 기능이나 개념을 Core에 혼입하지 않는다.

이 문서 전체를 새 창 첫 메시지로 붙여 넣은 뒤, 마지막에 한 줄만 추가하면 됩니다.

“위 인계문서를 AGEX 개발의 최상위 기준으로 적용하고, 현재 Repository 상태부터 확

인해서 Phase 0 구현을 시작해.”

그러면 새 창은 처음부터 설계 논의를 다시 반복하지 않고 실제 개발 모드로 바로 진입할 수 있습니다.